

AI ENGINEERING COHORT · DELIVERABLE 2 · WEEK 2

Design Opportunities

Retrieval, Context Construction, Storage & Index

Abhijeet Hiwale

July 2026

[research/week-2/design_opportunities.pdf](#)

Why These Three, and Why Now

Week 1 dealt with the components where the field is still arguing — admission, ranking, lifecycle. This week is the opposite: retrieval mechanics, context budgeting, and storage/index are largely *settled* questions in production practice as of mid-2026. That's a different kind of finding, and it changes how this week's review should go. There's less to debate and more to just decide. Eight ideas came out of the scan; seven are outright adopts. That's not me being lazy about the "defer/reject" options — it's that this is what a converged component looks like.

1. Hybrid Retrieval Isn't Optional Anymore

Problem: Deliverable 1 required semantic retrieval, but "semantic" alone isn't sufficient — pure dense vector search has a well-documented blind spot on exact terms, IDs, version numbers, and anything where the literal phrasing carries the meaning. Production write-ups this year put the failure rate of pure vector search at roughly 40% of real-world queries. That's not a tail case, that's a coin flip.

What I'd actually build: BM25 and dense vector search run in parallel, fused with Reciprocal Rank Fusion — which fuses on rank rather than raw score, sidestepping the score-incompatibility problem that breaks naive weighted averaging. This alone is reported to get typical systems to around 91% recall@10, and every modern vector store worth using already supports both retrieval paths natively, so this isn't exotic infrastructure — it's closer to turning on a setting.

Where I'd stop people from over-engineering this: cross-encoder or ColBERT reranking gets recommended everywhere, and it's a real technique — but it's not the first move. It's the move you make after you've benchmarked fusion alone and found a specific gap. I've decided to defer it explicitly rather than bundle it in now, because "add a reranker" has become the default answer to "retrieval isn't good enough" the way "add a cache" is the default answer to "the API is slow" — sometimes right, often reached for before it's actually needed.

The detail that's specific to memory (not generic RAG): Zep/Graphiti's production pipeline gets to good relevance ranking through fusion plus graph traversal, with zero LLM calls at retrieval time. That's the part worth internalizing — if retrieval quality depends on an LLM judging relevance on every turn, the system is expensive and slow in a way that compounds with every active user. Push that judgment into the index, not the retrieval call.

2. Context Rot Kills the "Just Use a Bigger Window" Instinct

Problem: It's tempting — especially with 2026-era million-token context windows — to treat the memory problem as basically solved by just having enough room. It isn't.

The finding that should end that argument: measured model performance starts degrading around the 32,000-token mark, even inside windows that are two orders of magnitude larger than that. More room to fill is more opportunity to fill it with the wrong information, not a bigger safety margin. I'm treating this as a design constraint rather than a mechanism — it doesn't tell you what to build, it tells you why selective retrieval isn't optional no matter how generous the model's context window gets.

The mechanism that follows from it: explicit, monitored, per-zone token budgets — system prompt, retrieved memory, conversation history, current input, output reserve — each with a hard ceiling instead of "whatever's left after everything else." One commonly cited split for a 128K window allocates roughly 40K to retrieved context and reserves fully half the window for output. The exact numbers are workload-specific, but the discipline of budgeting explicitly, per zone, with monitoring and alerts, is the transferable part.

The one most people miss: in any agentic use of this system, tool call outputs — not conversation history, not retrieved memory — are usually the actual budget killer. A few tool calls can burn more tokens than the whole visible conversation. I'm building tool-output accumulation in as its own explicitly budgeted zone from day one, rather than retrofitting it once it becomes a production incident.

3. Storage Is the Boring Decision, and That's the Right Call

Problem: Deliverable 1 needs a storage/index decision, and there's a real temptation to over-invest in this component because "vector database" sounds like the sophisticated part of the system.

The honest read after this week's scan: it isn't, and near every serious 2026 source agrees — under a few million vectors, on an existing Postgres install, pgvector is the right call. One system, transactional consistency with the rest of the application data, no new backup or monitoring stack. Move to a dedicated vector database only once real horizontal scale or strict per-tenant physical isolation is actually required, not preemptively.

The one place worth real design attention: tenant isolation. Namespace-based isolation is cheap and works fine at moderate scale, but it's an application-layer guarantee — a bug in query construction could leak across tenants. Physical per-tenant isolation is the harder guarantee and costs more. Given that Deliverable 1's constraints treat isolation as a hard requirement rather than a nice-to-have, I'm documenting the trigger for when to upgrade from namespace to physical isolation now, rather than deciding it under pressure later.

Full matrix of all eight ideas — including exact evidence and cost notes — is in *idea_evaluation_matrix.md*. Challenge notes from the live design review go in *challenge_notes.md* once that session happens.